

# Robotics Lab 2025: Homework 1

Bring up your robot

Nicholas Ruggiero: [https://github.com/Nicholasr567/Homework1\\_RL](https://github.com/Nicholasr567/Homework1_RL)

Giuseppe Arena: [https://github.com/PeppeAre/Homework1\\_RL](https://github.com/PeppeAre/Homework1_RL)

October 29, 2025

## Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Task 1: Modify URDF and Visualize in Rviz</b>                | <b>3</b>  |
| 1.1      | Task 1.a/b: Launch File and Rviz Configuration . . . . .        | 3         |
| 1.2      | Task 1.c: Collision Meshes and Movement . . . . .               | 3         |
| <b>2</b> | <b>Task 2: Add sensors and controllers and spawn in Gazebo</b>  | <b>6</b>  |
| 2.1      | Task 2.a: Spawning in Gazebo . . . . .                          | 6         |
| 2.2      | Task 2.b/c: Hardware Interface and Controller Loading . . . . . | 7         |
| <b>3</b> | <b>Task 3: Add a Camera Sensor</b>                              | <b>8</b>  |
| 3.1      | Task 3.a/b: Camera Link and Sensor XACRO . . . . .              | 8         |
| 3.2      | Task 3.c: Launch and Verify Topic . . . . .                     | 9         |
| <b>4</b> | <b>Task 4: ROS 2 Controller Node</b>                            | <b>10</b> |
| 4.1      | Task 4.a: Create Package and Node . . . . .                     | 10        |
| 4.2      | Task 4.b: JointState Subscriber . . . . .                       | 11        |
| 4.3      | Task 4.c/d: Position and Trajectory Publishers . . . . .        | 11        |

This document contains the Homework 1 of the Robotics Lab class, focused on simulating the "Armando" manipulator.

## 1 Task 1: Modify URDF and Visualize in Rviz

This task involved creating the necessary launch and configuration files to visualize the robot model in Rviz 2.

### 1.1 Task 1.a/b: Launch File and Rviz Configuration

We created the `armando_description/launch/armando_display.launch.py` file. This file loads the URDF, starts `robot_state_publisher`, `joint_state_publisher_gui`, and `rviz2`. We also created a `.rviz` configuration file to automatically load the model and passed it as an argument.

```
# Nodo Robot State Publisher
node_robot_state_publisher = Node(
    package='robot_state_publisher',
    # ...
    parameters=[{'robot_description': robot_desc, 'use_sim_time': False}]
)

# Nodo Joint State Publisher (con GUI per gli slider)
node_joint_state_publisher_gui = Node(
    package='joint_state_publisher_gui',
    # ...
)

# Nodo Rviz2 (con configurazione)
node_rviz2 = Node(
    package='rviz2',
    executable='rviz2',
    name='rviz2',
    output='screen',
    arguments=['-d', rviz_config_file] # Argomento per caricare il file .rviz
)
```

Listing 1: `src/armando_description/launch/armando_display.launch.py` (Key nodes)

The successful launch is shown in Figure 2 (with GUI) and 1 (without). Figure 3 shows the initial loaded state.

### 1.2 Task 1.c: Collision Meshes and Movement

The collision meshes in the URDF were substituted with primitive shapes. The sliders correctly control the joint states, as shown in Figure 4 where the robot is in a non-default pose.

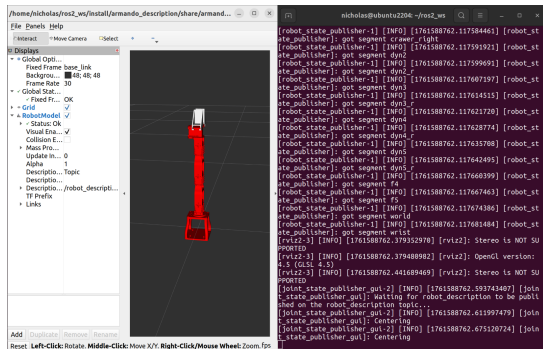


Figure 1: Rviz loaded without sliders.

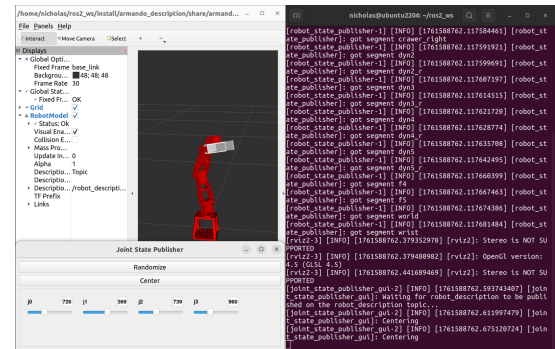


Figure 2: Rviz with joint\_state\_publisher\_gui active.

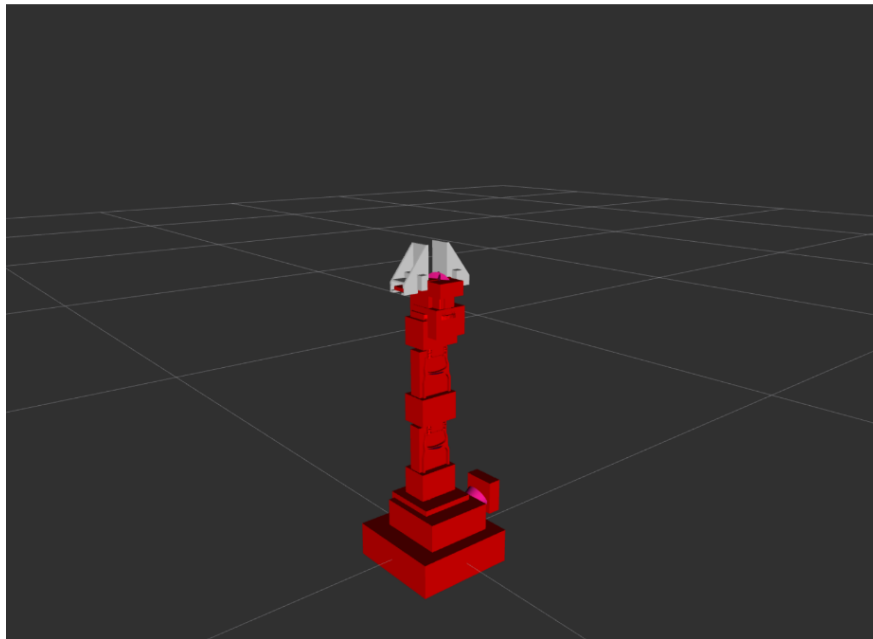


Figure 3: Robot visualized in Rviz with the loaded configuration.

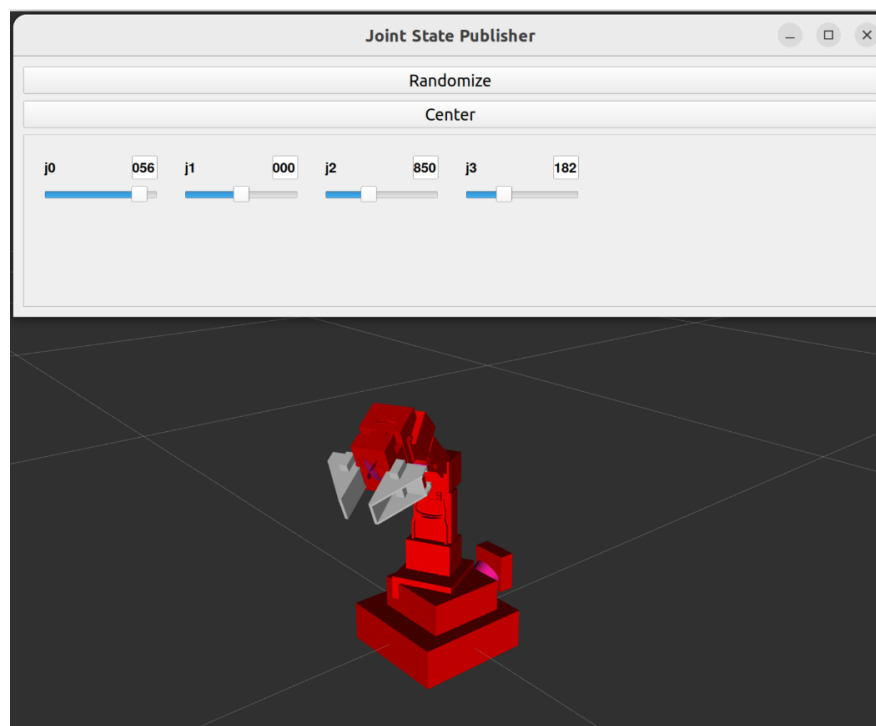


Figure 4: Robot moved to a new pose using the GUI sliders.

## 2 Task 2: Add sensors and controllers and spawn in Gazebo

This task focused on creating the Gazebo simulation environment and integrating the `ros2_control` framework.

### 2.1 Task 2.a: Spawning in Gazebo

We created `armando_gazebo` package and the `armando_world.launch.py` file. This launch file starts Gazebo and spawns the robot model.

```
# Include Gazebo launch file
gazebo = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(get_package_share_directory('ros_gz_sim'), 'launch', 'gz_sim.launch.py')
    ),
    launch_arguments={ 'gz_args': '-r empty.sdf' }.items()
)

# Spawn entity node
spawn_entity = Node(
    package='ros_gz_sim',
    executable='create',
    arguments=[ '-topic', 'robot_description',
                '-entity', 'armando',
                '-x', '0', '-y', '0', '-z', '0.1'],
    output='screen'
)
```

Listing 2: `src/armando_gazebo/launch/armando_world.launch.py` (Spawning)

The result of launching this file is shown in Figure 5.

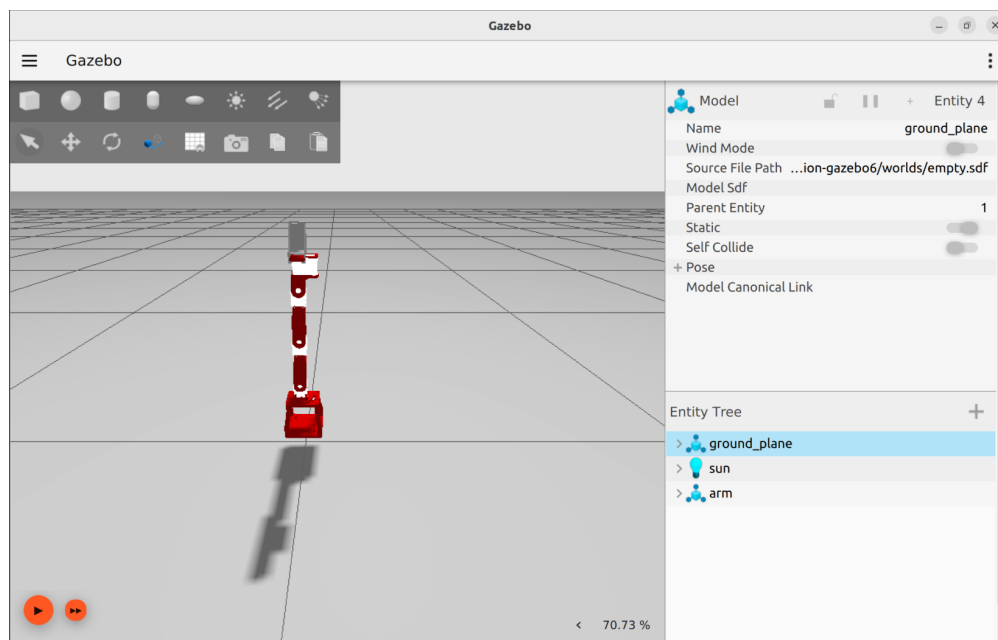


Figure 5: Armando robot spawned in the Gazebo world.

## 2.2 Task 2.b/c: Hardware Interface and Controller Loading

We added the `ros2_control` plugin to the URDF, defined controllers in `armando_controllers.yaml`, and configured the launch file to spawn them using `RegisterEventHandler`.

```
armando_controller_manager:
  ros__parameters:
    update_rate: 100  # Hz

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    joint_trajectory_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    simple_position_controller:
      type: forward_command_controller/ForwardCommandController
```

Listing 3: `src/armando_gazebo/config/armando_controllers.yaml`

```
# Spawner per il JointStateBroadcaster
spawn_joint_state_broadcaster = Node(
  package='controller_manager',
  executable='spawner',
  arguments=['joint_state_broadcaster', '--controller-manager', '/armando_controller_manager'
],
)

# Gestori di eventi per caricare i controller dopo lo spawn del robot
load_joint_state_broadcaster = RegisterEventHandler(
  event_handler=OnProcessExit(
    target_action=spawn_entity,
    on_exit=[spawn_joint_state_broadcaster],
  )
)

# ... (Event handlers for simple_position_controller and joint_trajectory_controller)
```

Listing 4: `src/armando_gazebo/launch/armando_world.launch.py` (Controller Spawners)

Figures 6 and 7 show the controllers being loaded and listed as active.

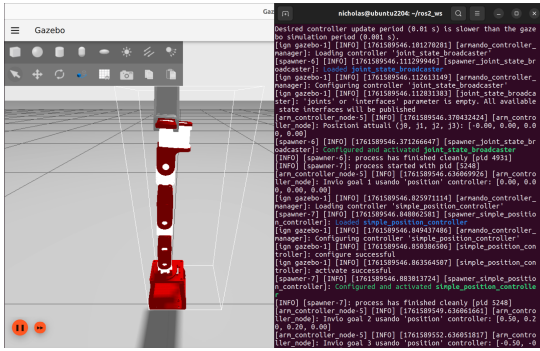


Figure 6: Log showing controllers loaded successfully.

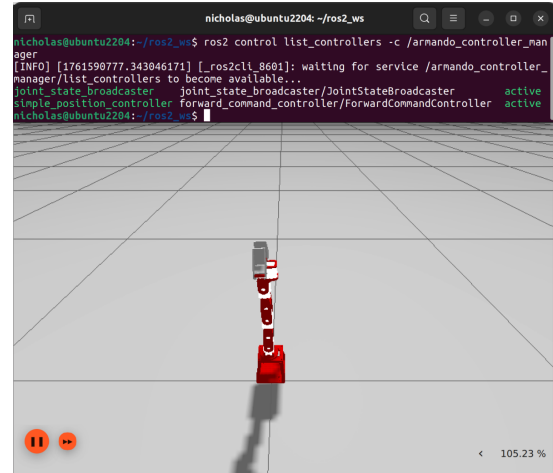


Figure 7: Listing active controllers via `ros2 control`.

### 3 Task 3: Add a Camera Sensor

This task involved adding a camera to the robot's URDF and bridging its data to ROS 2.

#### 3.1 Task 3.a/b: Camera Link and Sensor XACRO

We created `armando_sensors.xacro` to define the Gazebo sensor plugin and included it in the main URDF.

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="armando_sensors">

  <xacro:macro name="camera_sensor">

    <gazebo reference="camera_link">

      <sensor name="base_camera" type="camera">
        <pose>0 0 0 0 0 0</pose>

        <camera name="camera">
          <horizontal_fov>1.047</horizontal_fov>
          <image>
            <width>640</width>
            <height>480</height>
            <format>R8G8B8</format>
          </image>
          <clip>
            <near>0.1</near>
            <far>100</far>
          </clip>
        </camera>

        <topic>/camera</topic>
        <update_rate>30</update_rate>
        <always_on>1</always_on>
        <visualize>true</visualize>
      </sensor>
    </gazebo>
  </xacro:macro>
</robot>
```



```

    </sensor>
  </gazebo>
</xacro:macro>

</robot>

```

Listing 5: src/armando\_description/urdf/armando\_sensors.xacro

### 3.2 Task 3.c: Launch and Verify Topic

To make the Gazebo topic available in ROS 2, we added `ros_gz_bridge` to the launch file.

```

# --- Bridge per la telecamera ---
gz_ros_bridge = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
    arguments=[
        '/camera@sensor_msgs/msg/Image@gz.msgs.Image'
    ],
    remappings=[
        ('/camera', '/camera/image_raw')
    ],
    output='screen'
)

```

Listing 6: src/armando\_gazebo/launch/armando\_world.launch.py (Camera Bridge)

We verified the topic (Figure 8) and visualized the feed (Figure 9).

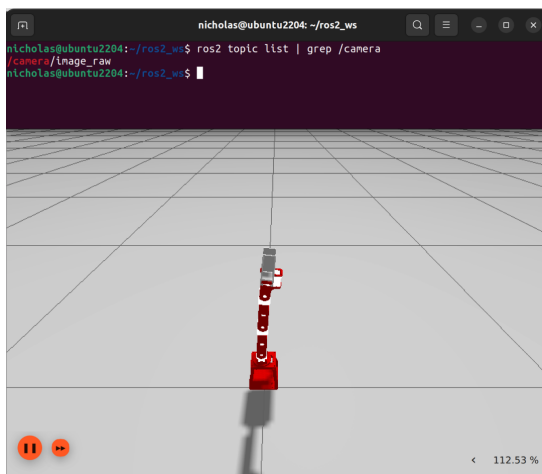
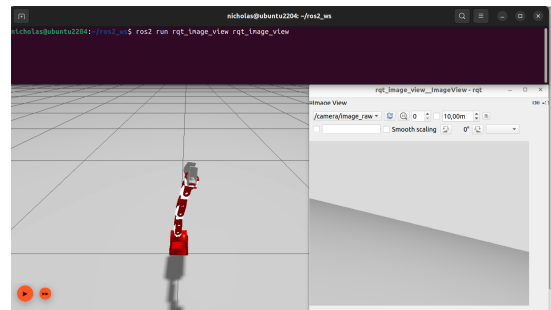
Figure 8: Verifying the `/camera/image_raw` topic.

Figure 9: Visualizing the camera feed in RQT.

## 4 Task 4: ROS 2 Controller Node

The final task was to create a C++ node to read joint states and send position commands.

### 4.1 Task 4.a: Create Package and Node

We created the `armando_controller` package and modified `CMakeLists.txt` to build the executable.

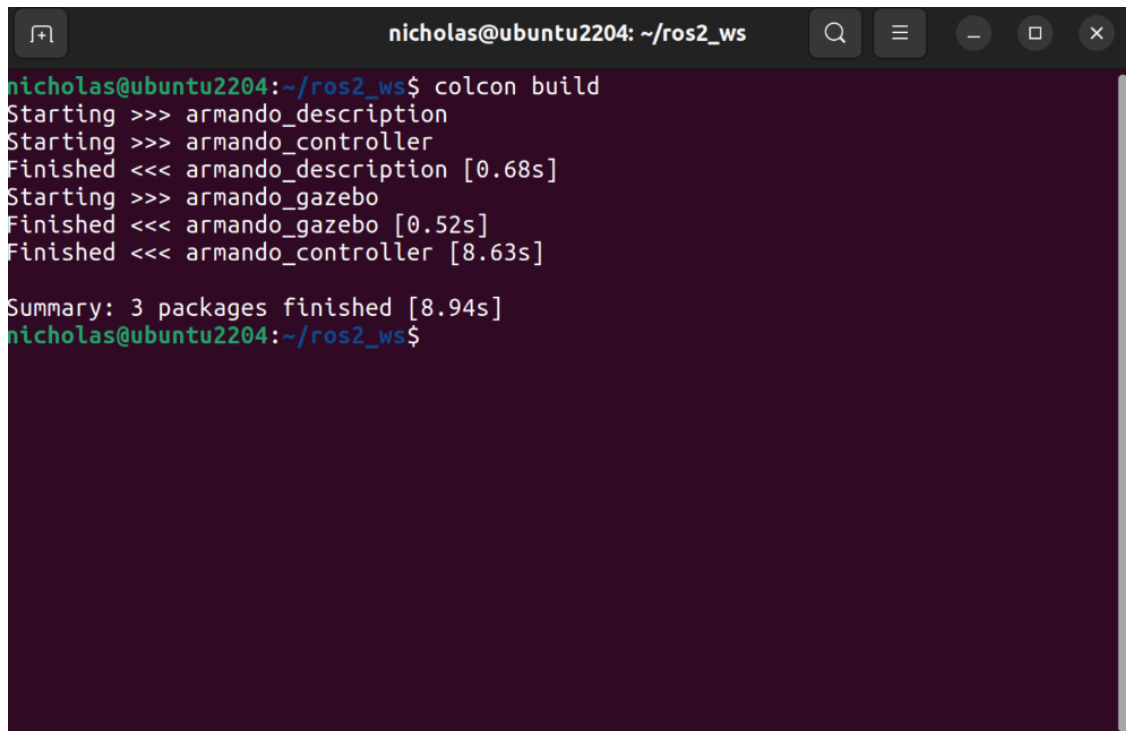
```
# (Punto 4a) Aggiungere l'eseguibile
add_executable(arm_controller_node src/arm_controller_node.cpp)

# (Punto 4a) Specificare le dipendenze
ament_target_dependencies(arm_controller_node
  rclcpp
  rclcpp_action
  sensor_msgs
  std_msgs
  control_msgs)

# Installare l'eseguibile
install(TARGETS
  arm_controller_node
  DESTINATION lib/${PROJECT_NAME})
)
```

Listing 7: `src/armando_controller/CMakeLists.txt`

The successful compilation is shown in Figure 10.



```
nicholas@ubuntu2204: ~/ros2_ws
nicholas@ubuntu2204:~/ros2_ws$ colcon build
Starting >>> armando_description
Starting >>> armando_controller
Finished <<< armando_description [0.68s]
Starting >>> armando_gazebo
Finished <<< armando_gazebo [0.52s]
Finished <<< armando_controller [8.63s]

Summary: 3 packages finished [8.94s]
nicholas@ubuntu2204:~/ros2_ws$
```

Figure 10: Successful `colcon` build of the workspace.

## 4.2 Task 4.b: JointState Subscriber

Inside the node, we created a subscriber to `/joint_states` and a callback to print the positions.

```
joint_state_sub_ = this->create_subscription<sensor_msgs::msg::JointState>(
    "joint_states", 10,
    std::bind(&ArmControllerNode::joint_state_callback, this, std::placeholders::_1));

// ...

private:
    void joint_state_callback(const sensor_msgs::msg::JointState::SharedPtr msg)
    {
        RCLCPP_INFO_ONCE(this->get_logger(), "Posizioni attuali (j0, j1, j2, j3): [%.2f, %.2f, %.2f, %.2f]",
            msg->position[0], msg->position[1], msg->position[2], msg->position[3]);
    }
```

Listing 8: `src/armando_controller/src/arm_controller_node.cpp` (Subscriber)

## 4.3 Task 4.c/d: Position and Trajectory Publishers

The node implements publishers for both `position_controller` and `joint_trajectory_controller`. A launch argument allows selecting which controller to use.

```
// Publisher per il controller di posizione (Punto 4c)
position_cmd_pub_ = this->create_publisher<std_msgs::msg::Float64MultiArray>(
    "simple_position_controller/commands", 10);

// Action Client per il controller di traiettoria (Punto 4d)
trajectory_action_client_ = rclcpp_action::create_client<FollowJointTrajectory>(
    this, "/joint_trajectory_controller/follow_joint_trajectory");

// ...

void publish_position_command(const std::vector<double>& positions)
{
    auto msg = std_msgs::msg::Float64MultiArray();
    msg.data = positions;
    position_cmd_pub_->publish(msg);
}

void send_trajectory_goal(const std::vector<double>& positions)
{
    // ... (check action server) ...
    auto goal_msg = FollowJointTrajectory::Goal();
    goal_msg.trajectory.joint_names = {"j0", "j1", "j2", "j3"};
    // ... (create point) ...
    trajectory_action_client_->async_send_goal(goal_msg);
}
```

Listing 9: `src/armando_controller/src/arm_controller_node.cpp` (Publishers)

The launch file includes the logic to select the controller based on the `controller_type` argument.

```
declare_controller_type_arg = DeclareLaunchArgument(
```

```

    'controller_type',
    default_value='position',
    description="Tipo di controller da usare: 'position' o 'trajectory'"
)
controller_type = LaunchConfiguration('controller_type')

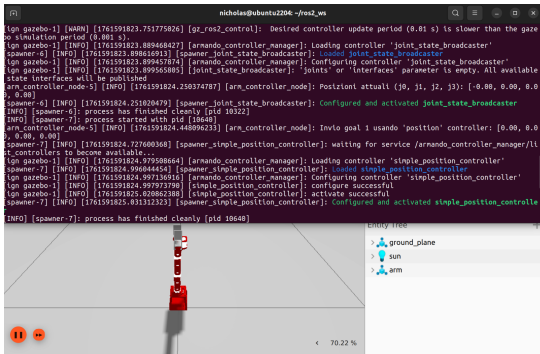
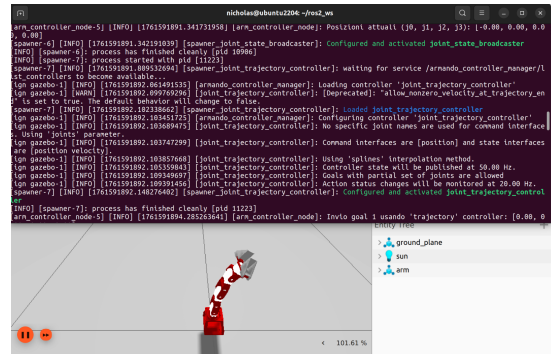
# Spawner per il SimplePositionController (Punto 4c)
spawn_simple_position_controller = Node(
    package='controller_manager',
    executable='spawner',
    arguments=['simple_position_controller', '--controller-manager', '/armando_controller_manager'],
    condition=IfCondition(PythonExpression(["", controller_type, " ' == 'position'"])))
)

# Spawner per il JointTrajectoryController (Punto 4d)
spawn_joint_trajectory_controller = Node(
    package='controller_manager',
    executable='spawner',
    arguments=['joint_trajectory_controller', '--controller-manager', '/armando_controller_manager'],
    condition=IfCondition(PythonExpression(["", controller_type, " ' == 'trajectory'"])))
)

```

Listing 10: src/armando\_gazebo/launch/armando\_world.launch.py (Controller Logic)

Figures 11 and 12 show the robot moving to commanded poses using both controller types.

Figure 11: Robot pose achieved via **Position Controller**.Figure 12: Robot pose achieved via **Trajectory Controller**.